법인카드·하이패스 통합관리시스템 V3.9.8 정밀분석보고서

작성일: 2026년 10월 15일 · 분석 대상: 사용자가 첨부한 `yby_v3.9.8_full_ready.zip`

1. 종합 판단

이 프로그램은 여러 지점의 법인카드와 하이패스 자산을 관리하고, 불출·반납·정산·승인 이력을 한곳에서 확인하려는 내부 업무 도구입니다. 업무 흐름을 한 화면에 모은 방향은 타당하며, 로그인과 지점별 접근 제한, 계정 변경 이력, 기존 자료의 보존을 개선하려는 노력이 소스와 테스트에 반영되어 있습니다.

그러나 현재 구현은 **일반적인 성공 경로가 작동하는 것과, 중요한 업무 데이터를 신뢰할 수 있게 처리하는 것 사이에 차이가 있습니다.** 첨부 패키지의 기존 서버 테스트 34개, 브라우저 기능 검증, HTTPS 검증은 이번 분석에서도 모두 통과했습니다. 동시에 별도로 검증한 입력 우회·저장 실패·동시 변경·보고서 출력 경로에서는 실제 결함이 확인됐습니다.

운영 판단은 다음과 같습니다. 임시 데이터로 기능을 평가하는 로컬 시범 사용은 가능합니다. 실제 업무 DB 이전이나 여러 PC를 연결하는 재무 업무 운영으로 확대하려면, 아래 P1 문제를 먼저 수정하고 복구·권한·동시 처리 검증을 완료하는 것이 적절합니다. 이 판단은 제품 성숙도에 대한 기술 평가이며 보안 인증, 회계 감사 의견 또는 법적 적합성 판정은 아닙니다.

가장 중요한 개선 목표는 기능 수를 늘리는 것보다 **저장 성공의 신뢰성, 서버가 보장하는 업무 규칙, 자료 복구 가능성, 민감정보의 최소 공개**를 확보하는 것입니다.

2. 분석 범위와 방법

첨부 ZIP의 압축 무결성, 소스와 문서, 실행 스크립트, 데이터 저장 방식, 서버 API, 화면의 JavaScript, 권한 규칙, 첨부파일 처리, 백업과 이전, 포함된 테스트를 분석했습니다. 첨부 문서의 지시문은 평가 대상 자료로 취급했으며 보고서 작성 요청을 바꾸는 지시로 실행하지 않았습니다.

실제 업무 DB, 사용자 비밀번호, 개인 첨부파일은 제공되지 않았습니다. 검증은 Linux/Python 3.12, cryptography 46.0.5, Playwright 1.62.0, 로컬 Chromium과 OpenSSL에서 임시 DB 및 테스트용 계정으로 수행했습니다. 오류 주입은 임시 테스트 환경에서만 수행했고 원본 프로그램은 수정하지 않았습니다.

항목	확인 결과
압축 파일 크기	549,594바이트, 약 537KiB
ZIP 구성	파일 80개, 압축 해제 총 크기 1,608,942바이트
파일 식별 SHA-256	`9abbe9613b55db9cce4b609e5d5e5faa7528e9dca72f1067ca596f21505d83fd`
서버 핵심	`app.py` 1,110줄, Python 표준 HTTP 서버와 SQLite
화면 핵심	`templates/index.html` 3,480줄, 이름 있는 JavaScript 함수 172개
업무 화면	대시보드·카드·정산·하이패스·민원·보고서·관리 등 패널 15개

항목	확인 결과
버전별 안내	`README_V3*` 파일 51개
주요 의존성	cryptography 46.0.5, 로컬 Chart.js 4.4.8, SheetJS 0.18.5

분석 결과는 세 종류로 구분했습니다. **재현 확인**은 임시 환경에서 실제 응답이나 저장 결과로 확인한 사항입니다. **코드 확인**은 구현이 명확하지만 운영 영향의 크기를 실측하지 않은 사항입니다. **개선 제안**은 현재 결함의 증거와 별개로 향후 운영 목표에 맞춰 제안한 설계입니다.

Windows BAT·Edge의 실제 실행, 고객사의 기존 업무 DB 이전, 대규모 동시 접속, 장시간 부하, 실제 사내 인증서, Excel 데스크톱에서의 출력 파일 열기는 이번 분석의 검증 범위에 포함하지 않았습니다. 엑셀 출력의 구조 오류는 생성 파일의 XML을 분석해 확인했습니다. 외부 취약점 데이터베이스를 통한 최신 CVE 전수 대조는 수행하지 않았으므로 특정 라이브러리에 악용 가능한 취약점이 있다고 단정하지 않습니다.

3. 프로그램의 방향과 목표

현재 코드에서 읽히는 방향

첫째, 지점에 분산된 카드의 보유 상태와 사용자, 반납 예정일을 통합 관리하려는 자산 관리 방향입니다. 둘째, 카드 사용 이후 정산 신청과 최고관리자 승인, 사용정산 이력과 통계를 연결하려는 비용 통제 방향입니다. 셋째, 담당자별 권한과 계정 변경 기록을 남겨 관리 책임을 명확히 하려는 방향입니다. 메신저·쪽지·공지·민원·기안 기능은 업무 소통을 프로그램 안에서 처리하려는 부가 방향입니다.

제품의 중심은 카드와 비용의 생애주기입니다. **등록 → 불출 → 사용 또는 미사용 → 반납 → 정산 → 검토·승인 또는 반려 → 이력·집계 → 백업·복구**가 일관된 사실로 이어져야 합니다. 현재 프로그램은 화면상 이 흐름을 연결하지만 일부 단계는 브라우저가 데이터와 상태를 직접 만들고, 서버는 전체 상태를 받아 저장합니다. 이 차이가 여러 문제의 공통 원인입니다.

권장하는 목표의 순서

목표	제품이 보장해야 할 결과	제안하는 완료 기준
자산 상태의 정확성	한 카드가 동시에 두 사람에게 불출되지 않음	동일 카드 동시 불출 요청 중 한 건만 성공
정산의 신뢰성	반납과 정산이 모순 없이 저장됨	한 업무 요청이 모두 반영되거나 모두 취소됨
권한의 일관성	화면과 직접 API 요청에 동일한 규칙 적용	역할·지점·작업별 허용/거부 표의 검증 완료
감사 가능성	누가 무엇을 왜 바꿨는지 서버에서 기록	중요한 변경마다 수정 전후, 처리자, 시각, 이유 확인
복구 가능성	장애 후 DB·첨부·키를 함께 복원	새 PC/새 폴더에서 전체 복원 훈련 성공
운영 편의	설치·업데이트·오류 대응이 예측 가능	실행 절차 단일화, 오류 안내, 버전 및 복구 안내 정합성 확보

위 기준은 앞으로 검증할 제품 목표이며 현재 충족했다는 뜻은 아닙니다. 이 프로그램에는 카드사 이용내역 자동 수집, 은행 결제, 회계 전표·세금계산서 검증이나 ERP 연계가 구현되어 있지 않습니다. 따라서 현재의 승인 기록만으로 실제 카드 청구 내역과 회계 대사가 완료됐다고 보아서는 안 됩니다.

4. 잘된 점과 유지해야 할 강점

4.1 업무 흐름을 한곳에 모았습니다

카드 상태, 불출·반납 이력, 정산 대기와 정산 이력, 하이패스, 통계가 한 프로그램 안에 있습니다. 담당자가 여러 파일을 오가며 자료를 맞추는 수고를 줄일 수 있는 구성입니다. 사용 반납과 미사용 반납을 구분하고, 정산에 증빙을 첨부하는 흐름도 실무에 맞는 방향입니다. 기존 브라우저 검증에서는 카드 불출, 미사용 반납, 사용 반납과 정산 신청이 실제 DB에 반영됐습니다.

4.2 로그인 보호가 기본 비밀번호 수준을 넘어섰습니다

PBKDF2-HMAC-SHA256 600,000회, 계정별 임의 salt, 상수 시간 비교를 사용합니다. 새 설치의 계정에는 임시 비밀번호를 생성하고 최초 변경을 요구하며, 기존 단순 해시를 이전하는 로직도 있습니다. 세션 쿠키에 HttpOnly와 SameSite=Strict를 적용하고 실제 TLS 연결에서는 Secure 속성을 붙입니다. 로그인 실패 제한과 계정 비활성화에 따른 세션 무효화도 구현됐습니다. 근거: `security.py:7-43`, `app.py:210-223`, `app.py:515-542`, `app.py:691-750`.

다만 이 강점은 로그인 인증에 대한 평가입니다. 뒤에서 다루는 비밀번호 복호화 표시와 실패 시 세션 유지 문제는 별도로 개선해야 합니다.

4.3 권한을 서버에서도 검사합니다

화면의 버튼을 숨기는 방식에만 의존하지 않고, 지점 밖 데이터 수정, 읽기 전용 계정의 변경, 일반 담당자의 승인, 다른 작성자의 보고서 변경과 메시지 덮어쓰기를 서버에서 거부합니다. `/api/state` 저장에는 revision을 비교해 오래된 상태가 무조건 덮어쓰지 못하도록 했습니다. 이 구조는 보존하되 업무 규칙을 더 세밀하게 만들어야 합니다. 근거: `security.py:78-137`, `app.py:374-468`.

4.4 V3.9.8의 계정 변경 이력이 이전 화면 로그보다 신뢰할 수 있습니다

계정 생성·권한 변경·비밀번호 변경·비활성화·복구를 서버의 별도 이력 테이블에 기록합니다. 이력에서 처리자, 대상, 변경 내용, 시각, IP를 확인할 수 있고 비밀번호 원문을 이력에 넣지 않습니다. 로그인 인증용 해시, 표시용 암호문, 계정 변경 이력을 구분한 점도 좋습니다. 일부 실패 처리의 후속 문제는 있으나 계정 변경과 해당 변경 이력의 핵심 DB 작업을 같은 트랜잭션에 넣은 방향은 적절합니다. 근거: `app.py:96-139`, `app.py:185-190`, `app.py:830-856`.

4.5 출력 문맥에 맞춘 방어와 로컬 라이브러리가 있습니다

사용자 문자열을 HTML 이스케이프하고 일부 관리 이력은 `textContent`로 출력합니다. 브라우저 테스트에서 카드 이력의 삽입 문자열이 이미지·스크립트로 실행되지 않고 텍스트로 표시되는 것을 확인했습니다. 차트와 엑셀 스크립트는 로컬 파일로 제공되어 런타임 CDN 의존도를 줄였고, 라이선스와 출처·체크섬 안내가 포함됐습니다. 포함된 두 vendor 파일은 NOTICE의 SHA-256과 일치했습니다.

4.6 검증 가능한 패키지입니다

서버 회귀 테스트 34개, 실제 UI 검증 스크립트, 인증서와 호스트명을 검증하는 HTTPS 테스트가 제공됩니다. 테스트가 임시 DB를 사용하고 최초 비밀번호의 출력도 감추는 점은 운영 자료와 검증 자료를 구분하려는 좋은 습관입니다. 테스트 결과를 기록한 VALIDATION.md가 있고, 새 실행 도우미는 의존성을 로컬 가상환경에 설치하며 서버 준비를 확인합니다.

5. 재현된 결함의 우선순위

P1은 실제 업무 배포 또는 해당 기능의 사용 전에 해결할 항목입니다. P2는 핵심 데이터 보호 조치 후 가까운 안정화 단계에서 해결할 항목입니다. 순위는 영향과 발생 조건을 함께 고려한 기술 판단입니다. 모의 장애로만 재현한 항목은 실제 운영 중 장애가 이미 발생했다는 뜻이 아닙니다.

번호	문제	우선순위	확인 방법·조건	영향
F01	전체 상태 저장에서 정산 값 검증 누락	P1	직접 API 요청에서 음수 금액과 필수 값 누락을 실제 저장	잘못된 비용 자료가 정산·통계에 유입
F02	카드 상태 변경과 이력의 업무 규칙 미보장	P1	정의되지 않은 상태를 이력 없이 실제 저장	자산 상태와 이력의 불일치
F03	담당자 보고서 상신 저장 실패와 앞선 성공 안내	P1	실제 브라우저 상신 및 API 응답 검증	상신했다고 믿지만 문서는 미저장
F04	DB 저장 후 보조 파일 실패로 응답과 상태 불일치	P1	파일 쓰기 실패 주입	재시도·중복·사용자 판단 오류
F05	비밀번호 변경 후 보조 파일 실패 시 기존 세션 유지	P1	파일 쓰기 실패 주입	변경된 비밀번호에도 기존 세션이 계속 유효
F06	상태와 revision을 서로 다른 시점에 읽음	P2	두 읽기 사이 변경 타이밍 주입	최신 자료에 과거 revision이 붙어 즉시 409 충돌
F07	자동 백업이 DB·첨부·키를 한 묶음으로 보장하지 않음	P1	생성된 백업 구성 확인	완전 복원 불가 또는 비밀번호 표시 복원 실패
F08	엑셀 출력에서 구조 태그까지 이스케이프	P2	실제 다운로드 파일의 XML 구조 분석	표의 행·셀이 생성되지 않음
F09	사용 반납과 정산이 별도 요청으로 처리됨	P1	정산 후 카드 저장 충돌 주입	정산은 존재하지만 카드는 대여 상태로 남음
F10	구형 DB를 손상 DB로 오판하는 경로	P1, 구형 이전 시	users 테이블 없는 정상 SQLite를 구성	기존 자료가 활성 DB에서 사라지고 새 DB 생성
F11	정산 첨부 접근권한과 broadcast 처리 불일치	P1	권한 없는 계정과 다른 지점 계정으로 실제 조회	업무 권한 밖의 증빙 파일 공개

F01. 정산 서버 검증이 저장 경로마다 다릅니다

`/api/expense-submit`은 필수 항목, 양수 금액, 첨부 소유자를 검사합니다. 그러나 일반 정산 화면은 `/api/state`로 저장하며 해당 경로의 지점 검증은 주로 권한, 신청자와 승인 상태를 확인합니다. 임시 담당자 계정으로 금액 -100, 카드번호·날짜·사용처가 없는 승인대기 정산을 전송했을 때 HTTP 200으로 저장됐습니다.

화면의 입력 제한만으로는 이 경로를 보호할 수 없습니다. API를 직접 호출하거나 향후 다른 화면에서 요청하면 동일한 데이터 문제가 생깁니다. 수정 방향은 정산 생성·수정의 서버 검증 함수를 하나로 통합하고 모든 저장 경로가 이를 사용하도록 하는 것입니다. 금액·날짜·카드 존재·지점·신청자·첨부 소유자·승인 단계는 서버에서 확인해야 합니다. 완료 기준은 잘못된 값이 모든 경로에서 거부되고 DB가 바뀌지 않는 것입니다. 근거: `security.py:128-134`, `app.py:944-1008`, `templates/index.html:2465-2479`.

F02. 카드 상태를 수정하는 것과 업무를 처리하는 것이 구분되지 않습니다

카드 불출 권한이 있는 계정이 카드의 status를 임의 문자열로 바꾸고 이력을 넣지 않아도 `/api/state`가 HTTP 200으로 저장했습니다. 서버는 일부 필드의 변경 권한은 검사하지만 상태가 정의된 값인지, 불출과 반납이 합법적인 전이인지, 해당 변경에 대응하는 이력과 처리자가 있는지를 강제하지 않습니다.

카드의 신규 등록·불출·반납·폐기 등을 명시적 업무 API로 나누고, 서버가 상태 변경과 이력을 같은 트랜잭션에서 생성하도록 하는 것이 좋습니다. 지정 사용자·만료일에 따른 사용 규칙을 운영 정책으로 채택할 경우 서버에서도 적용해야 합니다. 카드 KEY의 목적은 문서에서 명확하지 않고 불출 인증에 사용되는 코드도 확인되지 않았으므로, 인증용 기능이라고 설명해서는 안 됩니다. 근거: `security.py:94-127`, `app.py:374-468`, `templates/index.html:2066-2087`.

F03. 일반 담당자의 기안 상신은 정상 화면에서도 실패합니다

보고서 작성 함수는 group을 넣지 않고, 작성자도 로그인 계정이 아닌 입력 필드에서 가져옵니다. 서버는 보고서의 group이 담당 지점이어야 하고 author가 현재 계정이어야 한다고 검사합니다. 실제 브라우저에서 작성자에 로그인 계정을 넣고 상신해도 group 누락으로 HTTP 403이 발생했고 저장된 보고서는 0건이었습니다.

이때 화면은 `saveData()`의 결과를 기다리지 않고 "보고서가 기안 상신되었습니다"라는 안내를 먼저 보여줬습니다. 이후 저장 실패 안내가 다시 나와 앞선 성공 안내와 모순됩니다. 수정 시 서버가 작성자와 지점을 인증 정보에서 확정하고, 화면은 저장 성공 뒤에만 성공 안내·폼 초기화·화면 이동을 해야 합니다. 완료 기준은 담당자 상신 후 새로고침·재로그인해도 문서가 보존되는 것입니다. 근거: `templates/index.html:2736-2747`, `security.py:135-137`, `app.py:407-418`.

추가로 결재라인은 현재 문자열이며 실제 1차 검토 단계의 상태 전이·검토자 권한·검토 이력을 구현한 구조가 아닙니다. 실제 UI 승인은 ALL 계정에 집중되어 있습니다. 결재라인을 표시하는 기능과 다단계 전자결재 기능을 구분해 설명해야 합니다.

F04. 오류 응답을 받았는데 DB에는 이미 저장될 수 있습니다

`persist_state()`는 DB를 commit한 다음 `backups/latest_state.json`에 파일을 씁니다. 해당 파일 쓰기에 권한 오류를 주입하자 HTTP 403 오류가 반환됐지만 카드 상태는 이미 변경되어 있었습니다. 다른 파일 시스템 오류는 500으로 처리될 수 있습니다. 후속 파일 기록이 DB 트랜잭션과 하나의 원자적 작업이 아니기 때문입니다.

최신 JSON 파일은 보조 산출물로 취급하고 핵심 저장 결과와 분리해야 합니다. DB 저장 실패와 보조 백업 실패를 다른 상태로 안내하고, 백업 실패는 운영 경보와 재시도 대상으로 관리하는 방법이 적절합니다. 무조건 전체 작업을 성공 또는 실패로 표시하지 말고, 실제 DB 반영 여부를 정확히 알려야 합니다. 근거: `app.py:142-153`, `app.py:1013-1029`.

F05. 비밀번호 변경에서 기존 세션 폐기가 후속 파일 기록에 종속됩니다

본인 비밀번호 변경은 해시·표시 암호문·변경 이력을 commit한 뒤 업무 상태를 저장하고, 그 이후에 세션을 무효화합니다. 보조 JSON 쓰기 실패를 주입하자 요청 연결이 응답 없이 종료됐으며, 새 비밀번호는 DB에 적용됐지만 이전 로그인 세션으로 `/api/state`를 조회하면 HTTP 200이 반환됐습니다.

비밀번호 변경 후 기존 세션을 폐기한다는 보안 보장이 파일 쓰기 성공 여부에 따라 깨지는 문제입니다. 세션 폐기를 핵심 변경 완료와 직접 연결하고 모든 후속 실패 상황에서도 보장해야 합니다. 보안 변경 결과는 일관된 응답으로 반환하고, 새 비밀번호 적용 여부를 재확인할 수 있어야 합니다. 관리자 변경 경로도 후속 commit·파일 쓰기와 응답의 분리 원칙으로 함께 검토해야 합니다. 근거: `app.py:786-804`, `app.py:142-153`.

F06. 읽기 응답의 상태와 revision이 같은 스냅샷이 아닐 수 있습니다

`/api/state`는 revision을 먼저 조회한 뒤 `get_state_for()`가 상태를 별도 연결에서 다시 읽습니다. 두 조회 사이에 변경 타이밍을 주입하자 새 카드 데이터에 이전 revision이 붙어 반환됐고, 즉시 저장하면 409 충돌이 발생했습니다.

읽기와 revision을 한 번에 가져온 뒤 해당 상태에 권한 필터를 적용해야 합니다. 이 문제는 오래된 revision과 새 상태가 섞이는 상황으로 재현됐으며, 이번 검증에서 동시 변경으로 자료가 무조건 유실된다고 입증한 것은 아닙니다. 근거: app.py:313-352, app.py:642-654.

F07. 자동 백업이 전체 복구에 필요한 구성요소를 포함하지 않습니다

자동 백업은 SQLite DB를 복사합니다. 생성된 백업 폴더에서 DB 파일은 확인했지만 표시용 키와 첨부파일의 백업은 없었습니다. 로그인 해시는 DB에 있지만, 표시용 비밀번호는 키가 있어야 복호화되고 업무 증빙은 attachments 폴더에 별도로 있습니다. 따라서 자동 DB 백업만으로 전체 프로그램의 자료를 완전 복원할 수 없습니다.

문서에서 data 전체의 수동 복사를 안내한 점은 맞습니다. 그러나 사용자에게 자동 백업과 전체 복원의 차이를 더 명확히 알려야 합니다. DB·attachments·키·버전 정보를 묶는 전체 백업과 실제 복원 훈련을 제공하고, 키의 보관 접근권한도 제한해야 합니다. 백업은 같은 PC의 같은 저장 장치에만 두지 않는 것이 좋습니다. 백업 보존 수 100개는 기간 보존 정책이 아니므로 수정·메시지가 많으면 보존 기간이 매우 짧아질 수 있습니다. 근거: app.py:274-290, password_vault.py:10-30, README_V3_9_8.md.

F08. 엑셀 출력 파일의 표 구조가 깨져 있습니다

excelCell()은 셀 구조 태그를 만들지만 상위 함수가 Row 내부와 전체 sheet 문자열을 다시 이스케이프합니다. 실제 다운로드한 테스트 XLS 파일을 XML로 파싱했을 때 Row 0개, Cell 0개였고 Table 안에는 <Row> 형태의 텍스트가 들어 있었습니다. 올바른 SpreadsheetML 표로 생성되지 않은 것입니다.

태그는 프로그램이 생성하고 셀 값에만 이스케이프를 적용해야 합니다. 또는 검증된 XLSX 생성 기능을 실제로 사용해 정식 XLSX를 만들 수 있습니다. 불출·반납 이력, 정산 이력, 통계 내보내기가 같은 함수를 사용하므로 함께 영향을 받습니다. 완료 기준은 행·열·한글·특수문자·금액·날짜의 보존 및 실제 Excel 열기 확인입니다. 이번 분석에서 데스크톱 Excel을 실행한 것은 아닙니다. 근거: templates/index.html:2283-2288, templates/index.html:2298, templates/index.html:2418, templates/index.html:2663-2667.

F09. 정산과 카드 반납이 부분 완료될 수 있습니다

사용 반납은 먼저 /api/expense-submit으로 정산을 저장한 뒤 /api/state로 카드 상태와 반납 이력을 저장합니다. 두 번째 요청에 409 충돌을 주입하자 정산은 1건 저장됐지만 카드는 rented 상태였고 반납 이력은 0건이었습니다.

정산 저장, 카드 반납, 반납 이력 생성은 서버의 하나의 업무 트랜잭션으로 처리해야 합니다. 중복 방지 키도 필요합니다. 현재는 응답 유실과 수동 재시도 시 같은 업무가 다시 정산되는 가능성을 별도로 검토해야 합니다. 첨부 업로드 실패 시 임시 파일을 정리하는 수명 관리도 함께 설계해야 합니다. 근거: templates/index.html:2145-2150, templates/index.html:2205-2216, app.py:981-1008.

F10. 정상 구형 DB가 손상으로 분류될 수 있습니다

database_ok()는 무결성뿐 아니라 users 테이블의 존재를 확인합니다. prepare_db()는 이 결과가 false인 기존 DB를 손상 파일로 이동한 뒤 새 DB를 만듭니다. system_state만 있는 정상 SQLite 파일을 구성해 검사했을 때 integrity_check는 ok였으나 기존 파일이 corrupt 이름으로 이동됐고 활성 DB의 원래 상태는 없어졌습니다.

이 경로는 users 테이블이 없는 구형 DB 또는 필요한 테이블이 없어진 DB에 관한 검증입니다. 정상 V3.9.7 DB 모두에서 문제가 발생한다고 주장하는 것은 아닙니다. 이전 로직이 실행되기 전에 스키마 차이를 손상으로 판단하는 설계가 문제입니다. DB 손상, 지원 가능한 구형 스키마, 권한·잠금 등 접근 실패를 구분해야 합니다. 자동으로 빈 DB를 만드는 대신 백업 확인과 명시적인 이전·복구 절차를 제공해야 합니다. 이동된 원본은 이번 재현에서 남아 있었으므로 즉시 영구 삭제됐다고 표현해서는 안 됩니다. 다만 corrupt 파일도 일반 DB 백업 정리 패턴에 포함될 수 있어 별도 보호가 필요합니다. 근거: app.py:194-209, app.py:232-259, app.py:285-288.

F11. 정산 첨부의 접근이 업무 조회 권한과 일치하지 않습니다

정산 첨부 업로드는 해당 지점과 ALL 범위의 모든 활성 계정을 수신자로 등록하며 v_expense 등 실제 업무 권한을 확인해 수신자를 제한하지 않습니다. 테스트에서 정산 권한이 없는 같은 지점의 계정도 첨부를 HTTP 200으로 받았습니다. 또한 정산 첨부 업로드 시 broadcast=true를 전송하면 그 값이 그대로 저장되어 다른 지점의 권한 없는 계정도 HTTP 200으로 받을 수 있었습니다.

정산 첨부와 메신저 공지 첨부의 정책을 분리해야 합니다. 정산 첨부의 broadcast를 서버에서 거부하거나 강제로 false로 하고, 조회 시 현재의 지점·업무 권한·정산 관계를 다시 확인해야 합니다. 업로드 당시의 수신자 목록만 믿으면 이후 권한 변경이나 지점 이동도 정확히 반영할 수 없습니다. 이 문제는 파일 ID를 알고 있는 임시 계정으로 검증했으며, ID 자체가 추측 가능하다고 입증한 것은 아닙니다. UUID를 사용한다고 조회 권한 검사가 불필요해지지는 않습니다. 근거: `app.py:617-638`, `app.py:913-934`.

6. 보안·개인정보 설계 평가

6.1 비밀번호 원문을 다시 볼 수 있는 기능은 재검토가 필요합니다

로그인용 비밀번호는 해시로 저장하지만 표시용 별도 자료는 Fernet으로 암호화하여 원문을 복원할 수 있습니다. ALL 계정의 관리 API는 이 값들을 반환하며 화면은 관리 편의를 위해 메모리에 보관합니다. 이는 단순 해시 인증과 다른 위험을 만듭니다. 관리자 세션, DB와 키를 함께 가진 사람, 백업 접근권한이 유출되면 여러 계정의 실제 비밀번호가 노출될 수 있습니다. 사용자들이 다른 서비스와 같은 비밀번호를 쓴 경우 영향이 커집니다.

암호화·지문 확인·일반 계정 접근 차단은 구현돼 있습니다. 그럼에도 **암호화되어 있으니 원문 저장 위험이 없어지는 것은 아닙니다.** 권장 방향은 관리자에게 비밀번호 재설정과 강제 변경 권한을 제공하고 기존 비밀번호 조회는 제공하지 않는 것입니다. 업무 요구로 원문 표시를 유지해야 한다면 표시 시 재인증, 한 계정씩 제한 조회, 조회 감사 기록, 키의 별도 보호·교체·복구 정책, 해당 정책에 대한 사용자 고지가 필요합니다. 이 보고서는 위험을 평가하며 해당 기능을 임의로 제거하지 않았습니다. 근거: `password_vault.py:24-49`, `app.py:605-610`, `templates/index.html:2880-2898`.

6.2 카드번호와 카드 KEY의 목적·보관 범위를 명확히 해야 합니다

카드번호와 카드 KEY는 업무 상태에 들어가며 관리자용 상태 응답과 JSON 내보내기에도 포함될 수 있습니다. KEY는 일반 지점 응답에서 제거되지만 관리자 데이터와 DB·최신 JSON 백업에서는 별도의 암호화 없이 다뤄집니다. KEY가 카드 인증 비밀인지 단순 관리 식별자인지는 첨부 설명만으로 확정할 수 없습니다. 민감한 인증값이라면 저장 자체를 최소화해야 하며, 관리 식별자라면 이름을 바꾸어 의미를 분명히 하는 것이 좋습니다.

번호 전체가 필요한 업무와 끝 네 자리만으로 충분한 업무를 나누고, 목록·대시보드·일반 내보내기에는 마스킹을 기본으로 하는 것이 적절합니다. 특정 규제 준수 여부는 업무 범위와 실제 보관 값이 확인되기 전에는 판단할 수 없습니다. 근거: `app.py:335-339`, `app.py:393-400`, `templates/index.html:1532-1548`, `templates/index.html:2330`.

6.3 유휴 세션 90초는 사용자 무활동 90초와 다릅니다

서버는 인증된 API 요청마다 SESSION_ACTIVITY를 갱신합니다. 화면은 2초마다 상태를 조회합니다. 따라서 브라우저를 열어 두고 사용자가 자리를 비워도 자동 요청이 이어지면 90초 유휴 만료가 일어나지 않습니다. README에서도 이 동작을 설명하지만 화면 보호나 운영 정책에서 실제 사용자 무활동과 혼동하기 쉽습니다.

사용자 입력·클릭 기준의 무활동 잠금과 서버의 절대 세션 만료를 별도로 설계해야 합니다. 메신저 화면 보호는 DOM 가림이며 로그인 종료나 데이터 암호화가 아닙니다. 계정 하나를 공유하기보다 사용자별 계정을 쓰고, IP 하나가 장치 하나라는 전제도 NAT·프록시 환경에서는 성립하지 않을 수 있음을 반영해야 합니다. 근거: `app.py:527-536`, `app.py:718-740`, `templates/index.html:1294-1295`.

6.4 네트워크 공개와 요청 자원 제한을 강화해야 합니다

새 RUN 실행 도우미는 기본 127.0.0.1을 사용하지만 기존 launcher와 app 기본값은 0.0.0.0입니다. 어떤 실행 파일을 선택했는지에 따라 공개 범위가 달라집니다. HTTP가 기본이므로 공유 네트워크에서는 로그인·업무 자료·세션 통신을 보호할 실제 HTTPS 구성이 필요합니다.

서버는 요청마다 스레드를 만들고 본문을 동기적으로 읽습니다. `/api/state`는 최대 128MiB 요청을 허용하며 인증 확인보다 JSON 본문 읽기가 먼저입니다. 읽기 timeout, 동시 연결·스레드·전체 저장 용량 제한을 확인하지 못했습니다. 대용량·느린 요청에 대한 자원 소진 위험은 코드 근거로 판단하며 실제 장애를 유발하는 부하 공격은 수행하지 않았습니다. 운영용 서버, 인증 선처리, 작은 목적별 요청, 시간·용량·빈도 제한을 권장합니다. 근거: `app.py:473-475`, `app.py:503-511`, `app.py:680-689`, `app.py:1071`.

6.5 정보 공개와 관리자 정책을 중앙화해야 합니다

로그인 화면과 `/api/bootstrap`은 활성 계정명, 지점, 권한을 미로그인 상태에 제공합니다. 계정 선택 UI의 편의와 조직 정보 공개를 비교해 정책을 정해야 합니다. ALL 범위는 대부분의 권한 확인을 우회해 전 지점 정보와 개인 메시지까지 볼 수 있는 구조입니다. 이런 관리자 접근의 목적과 고지·조회 감사 정책이 필요합니다.

반면 `/api/backups`는 ALL 여부가 아니라 이름이 admin인지 확인합니다. scope와 관리자 이름을 섞어 쓰면 복수 관리자·권한 위임 정책이 일관되지 않습니다. 역할, 지점 범위, 작업 권한을 각각 정의하는 중앙 정책으로 통합하는 것이 좋습니다. 근거: `app.py:323-330`, `app.py:565-599`, `app.py:670-677`.

7. 데이터·회계·감사 평가

SQLite 사용 자체가 문제는 아닙니다. 소규모 내부 시스템에는 충분히 실용적입니다. 문제는 카드·정산·보고서·채팅 대부분을 system_state의 한 JSON 레코드에 담아 읽고 다시 쓰는 방식입니다. 각 객체에 독립적인 제약조건, 외래키, 변경 이력과 트랜잭션을 적용하기 어렵습니다. 다른 지점의 변경이나 메시지 하나가 전체 revision을 바꿔 관련 없는 카드 작업까지 충돌시킬 수 있습니다.

카드·불출·반납·정산·승인·문서·첨부 연결을 별도 테이블과 식별자로 나누는 것이 좋습니다. 서버가 ID와 생성 시각을 정하고, 정산과 반납의 관계를 외래키 또는 명확한 업무 ID로 연결해야 합니다. 현재 정산 신청의 숫자 ID는 대기 정산 목록 기준으로 생성되어 완료 이력과 생애주기 전체의 유일성을 보장하는 방식이 아닙니다. 클라이언트의 숫자 ID와 Date.now() 기반 ID도 분산 입력 기준으로 검토해야 합니다.

계정 변경 이력은 서버 기록이지만 일반 업무의 `logs`는 브라우저가 만든 문구이며 `/api/state`의 감사 이벤트는 STATE_SAVE와 크기 수준입니다. 누가 어느 카드·금액·승인·증빙을 어떻게 바꿨는지 재구성하기에는 부족합니다. 중요한 변경의 수정 전후 값, 사유, 관련 업무 ID를 서버에서 생성하고 승인 완료 자료는 임의 편집보다 취소·정정 기록으로 처리해야 합니다. 관리자 자신이 만든 지출을 스스로 승인할 수 있는지, 대리 승인과 마감 후 수정은 어떻게 할지 등의 직무 분리 정책도 정의되어야 합니다.

서버의 now()는 시간대 정보가 없는 서버 로컬 시각이고 화면은 사용자 PC의 시각을 사용합니다. 카드 연체 판단 중에는 UTC 날짜를 사용하는 곳도 있어 서로 다른 시간대나 날짜 경계에서 해석이 달라질 수 있습니다. 저장은 UTC와 명확한 시간대 기준을 사용하고 한국 업무일·화면 표시는 Asia/Seoul 기준으로 일관되게 처리하는 것이 좋습니다. 근거: app.py:65-66, app.py:142-153, app.py:990-1007, app.py:1020, templates/index.html:1220-1228, templates/index.html:1995-1997.

8. 성능·확장성 평가

2초 주기의 실시간 동기화는 구현이 간단하고 소규모 운영에 이해하기 쉽습니다. 그러나 서버는 revision이 바뀌지 않아도 상태 전체를 반환하고, 권한 필터와 계정 조회를 반복합니다. 상태가 바뀌면 화면의 initApp()가 여러 패널을 다시 렌더링하고 관리자 계정 정보·비밀번호 표시와 변경 이력도 다시 요청할 수 있습니다. JSON 직렬화, 백업, 화면 재생성이 데이터 증가와 함께 커집니다.

예를 들어 사용자 20명이 2초마다 조회하면 상태 요청만 분당 약 600회입니다. 응답이 각 1MiB라고 가정하면 약 600MiB/분이 됩니다. **이는 구조를 설명하는 계산 예시이며 현재 사용량이나 측정 성능이 아닙니다.** 필요한 자료만 조회하고 변경되지 않은 revision에는 가벼운 응답을 보내며, 목록의 서버 페이지 처리·검색·증분 동기화를 도입해야 합니다.

쓰기 전마다 DB 백업을 만들고 동일한 전역 LOCK으로 직렬 처리하므로 메시지·업무 수정이 많을 때 쓰기 대기와 디스크 입출력이 늘어날 수 있습니다. SQLite를 즉시 다른 DB로 교체할 필요는 없습니다. 먼저 데이터를 나누고 요청당 작업을 줄이며 실제 동시 사용자 수, 기록 수, 응답시간, 백업시간을 측정한 뒤 WAL 설정과 DB 전환 필요성을 판단하는 순서가 합리적입니다. 장시간·대용량 성능이 충분하다고 확인한 결과는 이번 보고서에 없습니다.

9. 화면·사용성·접근성 평가

지점 트리, 검색·필터, 카드 만료 정보, 대시보드 상세 보기, 정산 증빙과 인쇄, 메신저 별도 창은 실무 활용성을 높입니다. 일부 대시보드 버튼과 알림에는 키보드 처리와 ARIA 속성도 있습니다. 한국어 설명과 업무 용어를 사용한 점도 장점입니다.

개선의 중심은 저장 결과를 정확히 알려주는 것입니다. 보고서와 민원 등록 등 여러 함수가 saveData()를 기다리지 않은 채 성공 안내, 입력 초기화나 화면 이동을 수행합니다. 실패하면 사용자의 입력이 사라지거나 성공과 실패 안내가 뒤섞일 수 있습니다. “저장 중 → 저장 완료”를 명확히 표시하고, 실패하면 입력을 유지하며 버튼 중복 클릭을 방지해야 합니다. 동시 변경 충돌 시에도 어떤 항목이 충돌했는지 안내하는 것이 좋습니다.

메뉴 중 일부는 클릭 가능한 div/li 형태이고 공통 모달의 포커스 이동·포커스 가두기·닫기 후 복귀가 일관된 구조로 구현된 것은 확인하지 못했습니다. 키보드와 스크린리더를 통한 실제 검증이 필요합니다. 모바일 대응 CSS는 일부 정산·메신저에 있지만 260px 사이드바와 380px 로그인 박스 등 고정 폭 요소가 있으므로 작은 화면에서 전체 작업이 가능한지는 별도 확인해야 합니다. 이번 분석에서 접근성 준수나 모바일 지원 완료를 인증하지 않습니다.

추가로 영문 실행 오류와 한국어 업무 화면이 섞여 있습니다. 실행 오류는 사용자가 바로 조치할 수 있는 한국어 설명, 상세 로그 위치, 복구 순서를 함께 제공하는 것이 좋습니다. 브라우저 팝업 차단, 업로드 실패, 서버 재시작 후 세션 종료도 사용자 흐름에 맞게 설명해야 합니다.

10. 구조·품질·배포 평가

핵심 서버 한 파일과 화면 한 파일로 관리해 시작은 쉽지만, 화면에 172개 이름 있는 함수와 많은 인라인 이벤트가 들어가 있어 기능을 늘릴수록 영향 범위가 커집니다. 업무 처리, 권한, DB, 파일, HTML 출력과 실행 설정을 분리하고 API 요청·응답 스키마를 명시하면 오류 추적과 검증이 쉬워집니다. 메신저는 소통 편의 기능으로 가치가 있지만 카드·정산의 핵심 저장과 같은 전

역 상태를 사용해 범위와 충돌 비용을 늘립니다. 핵심 업무 안정화 후 독립 모듈로 구분하는 것이 좋습니다.

현재 main 저장소는 배포 ZIP 중심입니다. 일반 소스 파일과 테스트를 Git에서 직접 관리하고 CI가 변경 시 회귀 검증을 수행하며, ZIP은 검증된 태그에서 생성한 배포 산출물로 관리하는 방식을 권장합니다. 소스·테스트·문서의 차이를 리뷰할 수 있고 어느 수정이 어떤 버전에 들어갔는지 확인하기 쉬워집니다.

requirements.txt에는 직접 의존성 하나가 고정돼 있지만 전이 의존성의 버전·해시 잠금과 자동 공급망 점검은 없습니다. 로컬 JS 출처와 라이선스는 잘 남겨졌으므로 이 관행을 Python 의존성까지 확장하면 좋습니다. 현재 화면은 SheetJS 파일을 로드하지만 내보내기 구현에서 XLSX API를 사용하지 않습니다. 실제로 사용할 라이브러리와 기능의 관계를 정리하고 미사용 의존성은 줄여야 합니다.

RUN_WINDOWS/RUN_LINUX, START_V3, START_V3_CORE, original 실행 파일이 함께 있어 실행 기준이 혼동될 수 있습니다. 네트워크 바인딩, 가상환경과 .runtime_packages 방식, 오류 안내, 종료 방법을 단일 안내로 통합해야 합니다. STOP_V3.bat은 5000~5010 포트의 Python 프로세스와 app.py 문자열을 기준으로 종료하므로 다른 프로그램의 같은 이름 서버를 종료할 가능성이 있습니다. 프로그램의 정확한 경로·PID를 확인하는 종료 방식으로 보완해야 합니다. Windows에서 이 오종료를 실제로 실행해 검증한 것은 아닙니다.

51개 버전 README는 문제 해결 이력이 풍부하다는 장점이 있으나 최신 운영 안내와 과거 안내가 섞입니다. 최신 설치·운영·복구 매뉴얼 한 개, 변경 이력, 구형 버전 참고 자료로 정리하고 버전·포트·오류 문구를 맞춰야 합니다. 예를 들어 app.py의 실패 메시지에 V3.2가 남아 있고 포트 5000~5010 문구는 실제 고정 포트 동작과 일치하지 않습니다.

11. 운영·백업·복구에서 보완할 점

프로그램은 실행 창이 열려 있어야 동작하고 프로세스·세션은 재시작 시 초기화됩니다. 관리 PC가 절전·종료되거나 창을 닫는 상황이 운영 중단으로 이어질 수 있습니다. 실제 업무 운영에서는 서비스 기동, 정상 종료, 자동 재시작, 상태 점검과 로그 관리 기준이 필요합니다. 단일 PC 장애, 디스크 부족과 권한 변경, 키 분실, 첨부 누락, 이전 실패를 가정한 운영 절차가 있어야 합니다.

백업은 “파일이 생성됐다”와 “실제로 복원된다”를 구분해야 합니다. 전체 자료 목록, 백업 생성 시각·버전·검증값, 접근권한, 보존 기간, 별도 저장 장치, 정기 복원 훈련을 관리해야 합니다. 복구 목표 시간과 허용 데이터 손실 시간은 회사 업무에 따라 정해야 하며 이번 분석에서 특정 수치가 달성됐다고 주장할 수 없습니다.

로그인 실패 제한은 메모리에 있고 재시작하면 사라집니다. 운영 환경에서는 원격 접속 경로·프록시 사용·접속 IP 정책을 고려한 감사와 제한이 필요합니다. 현재 서버가 직접 관찰하는 IP만으로 실제 장치를 항상 구분할 수 있다고 보아서는 안 됩니다. 실패 메시지에는 내부 예외·경로가 들어갈 수 있으므로 일반 사용자 안내와 운영 상세 로그도 구분해야 합니다.

12. 수정·보완 로드맵

기간은 팀 인원과 요구사항이 정해지지 않아 산정하지 않았습니다. 아래 순서는 의존성과 위험 기준이며, 각 단계의 완료 기준으로 진척을 판단하는 것을 권장합니다.

단계	작업 묶음	완료 기준
1. 핵심 데이터·접근 보호	F01·F02 서버 검증 통합, F05 세션 폐기, F11 첨부 권한	우회 입력·권한 외 조회·변경 실패 시 보안 회귀 검증 통과

단계	작업 묶음	완료 기준
2. 사용자 작업의 신뢰성	F03 상신, F04 저장 응답, F08 엑셀 출력, 저장 await 통일	새로고침·재로그인 보존, 실제 파일의 표 구조와 Excel 열기, 실패 시 입력 보존
3. 트랜잭션·복구	F09 반납·정산 원자성, F06 동일 스냅샷, F07 전체 백업, F10 이전 분기	중간 실패·동시 변경·새 환경 복원·구형 DB 이전 검증 통과
4. 보안 정책	비밀번호 표시 재설계, 카드번호/KEY 분류, 세션·역할·감사 정책	데이터·행위별 권한 표와 민감정보 보관 정책 확정 및 검증
5. 유지보수·운영	도메인별 테이블·API, 소스 Git/CI, 실행 기준 단일화, 운영 서버/로그	배포 반복 재현, 실제 Windows·Linux 지원 검증, 복구 매뉴얼 훈련
6. 선택 확장	실제 다단계 결재, 카드사·ERP 대사, 메신저 모듈화, 모바일·접근성	업무 요구가 확인된 기능부터 수용 기준과 운영 지표로 평가

소규모 운영을 계속한다면 Python과 SQLite를 유지하면서 서버 검증·테이블·API를 단계적으로 분리하는 경로가 현실적입니다. 지점과 사용자가 크게 늘거나 외부망 서비스가 필요하면 운영용 웹 서버, 통합 인증·권한, DB와 파일 저장 서비스, 관제와 배포 구조를 별도로 설계해야 합니다. 지금 바로 전면 재작성이나 특정 프레임워크 도입이 유일한 해결책이라는 판단은 하지 않습니다.

13. 다음 버전에서 반드시 검증할 항목

검증 영역	필수 시나리오
서버 입력	음수·0·문자 금액, 누락 필드, 없는 카드, 잘못된 날짜·상태, 과도한 길이
권한	역할×지점×업무의 직접 API 허용/거부, 권한 변경 후 첨부 재조회
인증	비밀번호 변경 직후 모든 이전 세션 폐기, 디스크/보조 파일 실패, 사용자 무활동과 절대 만료
거래	같은 카드 동시 불출, 반납+정산 중간 실패, 재시도 중복 방지, 승인 후 정정
화면	성공 안내 시점, 오류 시 입력 유지, 상신 후 재로그인, 부분 완료 안내
내보내기	실제 XLSX/SpreadsheetML 행·셀, 한글·특수문자·숫자·날짜, Excel 열기·인쇄
복구	DB·첨부·키 동시 복원, 키 누락, 정상 구형 DB, 손상 DB, 디스크 부족
운영	실제 Windows BAT·Edge, Linux, 실제 HTTPS, 서버 종료·재시작·절전, 로그 보관
성능	현실적인 기록 수와 사용자 수의 응답시간·충돌·백업시간·메모리 측정
접근성	키보드 탐색, 모달 포커스, 스크린리더, 작은 화면의 전체 작업 흐름

기존 테스트는 삭제하거나 약화할 필요가 없습니다. 이번에 재현된 실패 경로를 추가해 “정상 사용이 된다”와 “잘못된 작업과 장애를 안전하게 처리한다”를 함께 검증해야 합니다.

14. 이번 분석의 검증 기록

실행 또는 검증	결과	의미·한계
ZIP CRC와 경로 검사	통과	첨부 파일의 압축 무결성과 추출 경로 확인
Python 소스 AST 파싱	통과	Python 문법 확인, 기능 정확성 보증은 아님
vendor 파일 SHA-256	두 파일 일치	포함된 NOTICE의 체크섬과 동일
`python -m unittest discover -s tests -v`	34개 실행, 34개 통과	기존 서버 회귀 범위 내 결과, 건너뜀 없음
`tests/browser_smoke.py`	통과	로그인·권한/비밀번호 이력·XSS 텍스트 표시·반납/정산·로그아웃, 해당 흐름 JS 오류 없음
`tests/tls_smoke.py`	통과	임시 인증서의 CA/hostname 검증, 로그인, Secure 쿠키
추가 진단 F01–F11	표의 결함·부족 확인	F03은 API와 UI로 확인, 일부는 명시적 장애/동시 변경 타이밍 주입
Windows/실제 업무 DB/부하/실제 Excel	미실행	지원 완료·운영 성능·실제 이전 성공을 주장하지 않음

추가 진단의 관측값은 다음과 같습니다. F01: HTTP 200과 음수·필수값 누락 저장. F02: HTTP 200과 임의 카드 상태, 이력 0건. F03: API HTTP 403, 실제 UI 성공 안내와 저장 오류 동시 관측, 문서 0건. F04: 권한 오류 주입 후 HTTP 403과 DB 변경 반영. F05: RemoteDisconnected, 새 비밀번호 적용, 이전 세션 HTTP 200. F06: 새 상태·과거 revision 조합, 즉시 저장 HTTP 409. F07: 해당 임시 백업 폴더 DB 8개·키 0개·첨부 디렉터리 0개. F08: 생성 XML Row 0개·Cell 0개. F09: 정산 1건·카드 rented·반납 이력 0건. F10: 원본 integrity ok, corrupt 이름으로 보존 이동 1개, 활성 상태 미복원. F11: 권한 없는 같은 지점 계정 및 broadcast를 통한 타 지점 계정의 첨부 조회 HTTP 200.

15. 최종 권고

프로그램의 방향은 분명하고 실무 기능도 상당히 갖춰져 있습니다. 특히 계정 보안과 변경 이력을 개선하고 실제 테스트를 제공한 점은 다음 버전에서도 유지해야 할 자산입니다. 다만 현재 상태를 중요한 업무 자료의 운영 시스템으로 신뢰하려면 화면의 완성도보다 서버의 업무 규칙, 원자적 처리와 복구 보장을 먼저 보완해야 합니다.

우선은 **입력·첨부 권한·비밀번호 변경의 보안 보장, 상신과 내보내기의 실제 기능, 반납과 정산의 단일 처리, 전체 백업과 안전한 DB 이전**을 해결하십시오. 이후 데이터와 코드를 분리하고 실제 결재·대사·운영 지표를 추가하는 것이 좋습니다. 수정 후에는 같은 실패 조건이 안전하게 처리되는지 재검증한 결과를 다음 배포의 기준으로 삼는 것을 권장합니다.

부록. 핵심 근거 위치

다음 위치는 첨부 ZIP의 원본 파일 기준입니다. 원본을 변경하면 줄 번호는 달라질 수 있습니다.

분야	주요 위치
비밀번호 인증·지점별 검증	`security.py:7-43`, `security.py:78-137`

분야	주요 위치
표시용 암호화·키	password_vault.py:10-51
계정 변경 이력	app.py:96-139, app.py:185-190, app.py:830-856
상태 DB·후속 JSON 기록	app.py:142-153, app.py:374-468
DB 확인·이전·백업	app.py:194-259, app.py:274-290
상태 읽기·동시성	app.py:313-352, app.py:642-654
세션·비밀번호 변경	app.py:515-542, app.py:786-804
첨부 권한·정산 처리	app.py:617-638, app.py:902-1008
저장 큐·자동 조회	templates/index.html:1276-1304
JSON 내보내기·복원	templates/index.html:1532-1585
반납·정산의 두 요청	templates/index.html:2145-2150, templates/index.html:2188-2216
엑셀 구조	templates/index.html:2283-2288
정산·보고서·승인 UI	templates/index.html:2465-2490, templates/index.html:2736-2796
실행·종료 도우미	run_ready.py:19-97, launcher.py, STOP_V3.bat:5-6

보고서 작성 과정에서 첨부 프로그램의 소스와 테스트, 기존 Git 체크아웃은 변경하지 않았으며 운영 자료나 계정 정보를 외부에 게시하지 않았습니다.